DD_PROV-INT-01 - Provisioning Broadcast and Reconciliation - Developer Implementation Guide

Version: 1.0 Status: Draft for review Bundle: 06_fulfillment Audience: mid-level backend developers, integration developers, QA Parent DD: DD_PROV-INT-01 - Provisioning Broadcast and Reconciliation

1. How To Read This Guide

This guide follows the execution flow.

FUL modules decide what technical action is needed. PROV-INT records and dispatches the actual vendor command.

Use this guide when implementing:

- activation network commands from FUL-03;
- restriction or suspension commands from FUL-04;
- termination/revocation commands from FUL-05;
- reconciliation between BSS desired state and network observed state;
- NOC force-sync.

2. Step 1 - Configure A Provisioning Target

Before dispatching commands, configure the external platform.

Example: Huawei GPON controller for Kenya.

```
insert into provisioning_target (
  target_id,
  operator_code,
  target_code,
  target_type,
  region_code,
  status,
  base_url_ref,
  secret_ref,
  capabilities_json,
  created_at,
  updated_at
) values (
  'tgt-huawei-nce-ke',
  'WIK',
  'HUAWEI_NCE_GPON_KE',
  'GPON_NMS',
  'NRB',
  'ACTIVE',
  'cfg/huawei-nce-ke/base-url',
  'secret/prov/huawei-nce-ke',
  '{"commands":["ACTIVATE_SERVICE","MODIFY_SERVICE","SUSPEND_SERVICE","RESUME_SERVICE","REVOKE_SERVICE"]}',
  now(),
  now()
);
```

Meaning:

- `target_code` is what NOC sees.
- `secret_ref` points to credentials in secret manager.
- `capabilities_json` lets validation reject unsupported command types early.

3. Step 2 - Configure Adapter Mapping

PLM service catalog has `provisioner_key` values such as `gpon-res`, `voip-sip`, and `verimatrix-iptv`.

Map each key to a target and adapter.

```
insert into provisioning_adapter_config (
```

```
  adapter_config_id,
  operator_code,
  provisioner_key,
  target_id,
  adapter_class,
  execution_mode_default,
  timeout_ms,
  max_retry_count,
  retry_policy_json,
  status
) values (
  'pac-gpon-wik',
  'WIK',
  'gpon-res',
  'tgt-huawei-nce-ke',
  'HuaweiNceGponAdapter',
  'SYNC_REQUIRED',
  25000,
  5,
  '{"baseSeconds":30,"factor":2,"maxSeconds":900}',
  'ACTIVE'
);
```

FUL-03 does not need to know the vendor class. It passes `provisionerKey = gpon-res`, and PROV-INT resolves the adapter config.

4. Step 3 - FUL-03 Sends Activation Command

FUL-03 has already:

1. Received `POST /api/service-fulfillment/execute`.
2. Loaded the package from SIP-01.
3. Loaded service definitions from PLM-CFG-01.
4. Loaded HomePass endpoints from RLM-CFG-01.
5. Resolved target node and port.

For internet activation, FUL-03 now calls PROV-INT:

```
POST /api/provisioning/commands
Idempotency-Key: FUL03:SUB-700045:svc_INET_100M_GPON:activate
Content-Type: application/json

{
  "operatorCode": "WIK",
  "sourceModule": "FUL-03",
  "sourceRefId": "flb-activate-001",
  "subscriptionId": "SUB-700045",
  "customerId": "CUS-100045",
  "homepassId": "HP-300045",
  "serviceRef": "svc_INET_100M_GPON",
  "provisionerKey": "gpon-res",
  "commandType": "ACTIVATE_SERVICE",
  "targetNodeCode": "OLT-NRB-KAREN-02",
  "targetPort": "0/4/07",
  "executionMode": "SYNC_REQUIRED",
  "payload": {
    "subscriberKey": "SUB-700045:INET",
    "equipmentId": "EQP-ONT-42",
    "speedDownMbps": 100,
    "speedUpMbps": 20
  }
}
```

PROV-INT returns:

```
{
  "commandId": "pcmd-20260531-000001",
  "status": "SUCCEEDED",
  "vendorCommandId": "NCE-CMD-8801",
  "adapterRefs": {
    "provisioningProfileId": "prf-100m-v3"
  }
}
```

FUL-03 then records its own `fulfillment_service_outcome` row as SUCCEEDED with `adapter_refs.commandId = pcmd-20260531-000001`.

5. Step 4 - PROV-INT Stores Command Before Dispatch

When PROV-INT receives the request, it first validates:

Validation	Error
`operatorCode` present	`PROV_MISSING_OPERATOR`
`provisionerKey` maps to active adapter config	`PROV_UNKNOWN_PROVISIONER_KEY`
target active	`PROV_TARGET_INACTIVE`
command type supported	`PROV_COMMAND_NOT_SUPPORTED`
idempotency key present	`PROV_IDEMPOTENCY_REQUIRED`
payload has subscriber key	`PROV_MISSING_SUBSCRIBER_KEY`

Then it inserts `provisioning_command`:

```
insert into provisioning_command (
  command_id,
  operator_code,
  idempotency_key,
  source_module,
  source_ref_id,
  subscription_id,
  customer_id,
  homepass_id,
  service_ref,
  command_type,
  target_id,
  target_node_code,
  target_port,
  payload_json,
  execution_mode,
  status,
  created_at,
  updated_at
) values (
  'pcmd-20260531-000001',
  'WIK',
  'FUL03:SUB-700045:svc_INET_100M_GPON:activate',
  'FUL-03',
  'flb-activate-001',
  'SUB-700045',
  'CUS-100045',
  'HP-300045',
  'svc_INET_100M_GPON',
  'ACTIVATE_SERVICE',
  'tgt-huawei-nce-ke',
  'OLT-NRB-KAREN-02',
  '0/4/07',
  '{"subscriberKey":"SUB-700045:INET","speedDownMbps":100,"speedUpMbps":20}',
  'SYNC_REQUIRED',
  'RECEIVED',
  now(),
  now()
);
```

Only after this row exists may the dispatcher call the vendor adapter.

6. Step 5 - Dispatch To Adapter

The dispatcher loads the adapter config:

```
select *
from provisioning_adapter_config
where operator_code = 'WIK'
  and provisioner_key = 'gpon-res'
  and status = 'ACTIVE';
```

Then it calls the adapter class.

Pseudo-code:

```
$command = $commands->lockForDispatch($commandId);
$config = $adapterConfigs->findForProvisionerKey($command->operator_code, $request->provisionerKey);
$adapter = $adapterRegistry->make($config->adapter_class);

$attempt = $attempts->start($command, $config);

try {
    $result = $adapter->dispatch($command);
    $attempts->markSuccess($attempt, $result);
    $commands->markSucceeded($command, $result);
} catch (RetryableProvisioningException $e) {
    $attempts->markRetryable($attempt, $e);
    $commands->markRetryable($command, $e);
} catch (FinalProvisioningException $e) {
    $attempts->markFinalFailure($attempt, $e);
    $commands->markFinalFailure($command, $e);
}
```

Adapter result shape:

```
{
  "success": true,
  "vendorCommandId": "NCE-CMD-8801",
  "adapterRefs": {
    "provisioningProfileId": "prf-100m-v3"
  },
  "observedProfile": {
    "speedDownMbps": 100,
    "speedUpMbps": 20,
    "status": "ACTIVE"
  }
}
```

7. Step 6 - Record Command Attempt

Insert one attempt row per vendor call.

```
insert into provisioning_command_attempt (
  attempt_id,
  command_id,
  attempt_no,
  adapter_class,
  status,
  request_payload_redacted,
  response_payload_redacted,
  vendor_status_code,
  duration_ms,
  created_at
) values (
  'pcma-20260531-000001',
  'pcmd-20260531-000001',
  1,
  'HuaweiNceGponAdapter',
  'SUCCESS',
  '{"subscriberKey":"SUB-700045:INET","speedDownMbps":100}',
  '{"commandId":"NCE-CMD-8801","profile":"100M"}',
  'OK',
  1200,
  now()
);
```

Then update the command:

```
update provisioning_command
set status = 'SUCCEEDED',
    vendor_command_id = 'NCE-CMD-8801',
    updated_at = now()
where command_id = 'pcmd-20260531-000001';
```

8. Step 7 - Update Desired State

After a command succeeds, PROV-INT writes or updates desired state.

```
insert into provisioning_desired_state (
  desired_state_id,
```

```
  operator_code,
  subscription_id,
  customer_id,
  homepass_id,
  service_ref,
  target_id,
  subscriber_key,
  desired_status,
  desired_profile_json,
  source_module,
  source_ref_id,
  effective_from,
  updated_at
) values (
  'pds-20260531-000001',
  'WIK',
  'SUB-700045',
  'CUS-100045',
  'HP-300045',
  'svc_INET_100M_GPON',
  'tgt-huawei-nce-ke',
  'SUB-700045:INET',
  'ACTIVE',
  '{"speedDownMbps":100,"speedUpMbps":20,"targetPort":"0/4/07"}',
  'FUL-03',
  'flb-activate-001',
  now(),
  now()
)
on conflict (operator_code, subscription_id, service_ref, target_id)
do update set
  desired_status = excluded.desired_status,
  desired_profile_json = excluded.desired_profile_json,
  source_module = excluded.source_module,
  source_ref_id = excluded.source_ref_id,
  effective_from = excluded.effective_from,
  updated_at = now();
```

Desired state is what reconciliation will compare against later.

9. Step 8 - Handle Duplicate Command

If FUL-03 retries the same activation because of a network timeout between FUL and PROV-INT, the same idempotency key arrives again:

```
FUL03:SUB-700045:svc_INET_100M_GPON:activate
```

PROV-INT must return the existing command outcome.

If existing command is `SUCCEEDED`, return the success response.

If existing command is `DISPATCHING`, return:

```
409 Conflict
```

```
{
  "errorCode": "PROV_COMMAND_IN_PROGRESS",
  "commandId": "pcmd-20260531-000001"
}
```

If existing command failed retryable, allow the caller to read it or call retry. Do not create a second command with the same key.

10. Step 9 - Broadcast Multiple Commands

For triple-play activation, FUL-03 may call PROV-INT once per service:

Service	Command type	Provisioner key	Target
Internet	`ACTIVATE_SERVICE`	`gpon-res`	Huawei NCE
Voice	`ACTIVATE_SERVICE`	`voip-sip`	VoIP switch
IPTV	`ACTIVATE_SERVICE`	`verimatrix-iptv`	IPTV middleware

Recommended approach:

1. FUL-03 keeps orchestration order and compensation rules.
2. PROV-INT handles each command idempotently.
3. FUL-03 records each service outcome using the returned `commandId`.

Do not make PROV-INT own FUL-03 compensation. It can expose inverse commands, but FUL-03 decides when compensation is required.

11. Step 10 - FUL-04 Sends Restriction Command

For a non-payment voice bar, FUL-04 calls:

```
POST /api/provisioning/commands
Idempotency-Key: FUL04:SUB-700045:svc_VOIP_STD:OUTGOING_VOICE_BARRED:add
Content-Type: application/json

{
  "operatorCode": "WIK",
  "sourceModule": "FUL-04",
  "sourceRefId": "flb-restrict-001",
  "subscriptionId": "SUB-700045",
  "customerId": "CUS-100045",
  "homepassId": "HP-300045",
  "serviceRef": "svc_VOIP_STD",
  "provisionerKey": "voip-sip",
  "commandType": "APPLY_RESTRICTION",
  "targetNodeCode": "VOIPSWITCH-NRB-01",
  "targetPort": "5012",
  "executionMode": "SYNC_REQUIRED",
  "payload": {
    "subscriberKey": "SUB-700045:VOICE",
    "restrictionCode": "OUTGOING_VOICE_BARRED",
    "fulfillmentAction": "IMMEDIATE",
    "remainingActiveRestrictions": ["OUTGOING_VOICE_BARRED"]
  }
}
```

Adapter applies the voice restriction and returns:

```
{
  "commandId": "pcmd-20260531-000101",
  "status": "SUCCEEDED",
  "vendorCommandId": "SIP-BAR-77123",
  "adapterRefs": {
    "sipBarringId": "bar-77123"
  }
}
```

Desired state should reflect restricted service:

```
{
  "desiredStatus": "RESTRICTED",
  "desiredProfile": {
    "restrictionCode": "OUTGOING_VOICE_BARRED",
    "voiceOutgoing": "BARRED"
  }
}
```

12. Step 11 - FUL-05 Sends Termination Command

For subscription termination, FUL-05 calls:

```
POST /api/provisioning/commands
Idempotency-Key: FUL05:SUB-700045:svc_INET_100M_GPON:revoke

{
  "operatorCode": "WIK",
  "sourceModule": "FUL-05",
  "sourceRefId": "flb-terminate-001",
  "subscriptionId": "SUB-700045",
  "customerId": "CUS-100045",
  "homepassId": "HP-300045",
  "serviceRef": "svc_INET_100M_GPON",
  "provisionerKey": "gpon-res",
  "commandType": "REVOKE_SERVICE",
  "targetNodeCode": "OLT-NRB-KAREN-02",
```

```
  "targetPort": "0/4/07",
  "executionMode": "SYNC_REQUIRED",
  "payload": {
    "subscriberKey": "SUB-700045:INET",
    "equipmentDisposition": "PENDING_COLLECTION",
    "reasonCategory": "CUSTOMER_REQUESTED"
  }
}
```

On success, desired state becomes:

```
{
  "desiredStatus": "NOT_PRESENT",
  "desiredProfile": {
    "reasonCategory": "CUSTOMER_REQUESTED",
    "terminatedAt": "2026-05-31T10:30:00Z"
  }
}
```

13. Step 12 - Retry Failed Commands

The retry worker loads commands:

```
select *
from provisioning_command
where status in ('FAILED_RETRYABLE', 'TIMED_OUT')
order by updated_at asc
limit 100;
```

For each command:

1. Check retry count from `provisioning_command_attempt`.
2. Check adapter config `max_retry_count`.
3. Check next retry time based on backoff.
4. Dispatch again.
5. Record another attempt.

Do not retry `FAILED_FINAL` automatically.

Example command:

```
prov:retry-failed-commands
```

Schedule:

```
every 1 minute
```

This is a Laravel scheduled command or queue worker, not necessarily a separate microservice.

14. Step 13 - Run Reconciliation

NOC or scheduler starts:

```
POST /api/provisioning/reconciliation-runs
Content-Type: application/json

{
  "operatorCode": "WIK",
  "targetCode": "HUAWEI_NCE_GPON_KE",
  "scopeType": "REGION",
  "scopeValue": "NRB"
}
```

PROV-INT creates:

```
insert into provisioning_reconciliation_run (
  run_id,
  operator_code,
  target_id,
  scope_type,
  scope_value,
  status,
  desired_count,
  observed_count,
```

```
  mismatch_count,
  started_at
) values (
  'prr-20260531-000001',
  'WIK',
  'tgt-huawei-nce-ke',
  'REGION',
  'NRB',
  'RUNNING',
  0,
  0,
  0,
  now()
);
```

Then the worker:

1. Loads desired state rows for the target/scope.
2. Calls adapter `fetchObservedState(scope)`.
3. Upserts `provisioning_observed_state`.
4. Compares desired and observed.
5. Creates reconciliation items for mismatches.
6. Completes the run.

15. Step 14 - Compare Desired And Observed State

Example desired state:

```
{
  "subscriberKey": "SUB-700045:INET",
  "desiredStatus": "ACTIVE",
  "desiredProfile": {
    "speedDownMbps": 100,
    "speedUpMbps": 20,
    "targetPort": "0/4/07"
  }
}
```

Example observed state:

```
{
  "subscriberKey": "SUB-700045:INET",
  "observedStatus": "ACTIVE",
  "observedProfile": {
    "speedDownMbps": 50,
    "speedUpMbps": 10,
    "targetPort": "0/4/07"
  }
}
```

Mismatch:

```
PROFILE_MISMATCH
```

Insert reconciliation item:

```
insert into provisioning_reconciliation_item (
  item_id,
  run_id,
  target_id,
  subscription_id,
  service_ref,
  subscriber_key,
  mismatch_type,
  severity,
  desired_snapshot_json,
  observed_snapshot_json,
  status,
  created_at
) values (
  'pri-20260531-000001',
  'prr-20260531-000001',
  'tgt-huawei-nce-ke',
  'SUB-700045',
  'svc_INET_100M_GPON',
  'SUB-700045:INET',
```

```
  'PROFILE_MISMATCH',
  'HIGH',
  '{"speedDownMbps":100,"speedUpMbps":20}',
  '{"speedDownMbps":50,"speedUpMbps":10}',
  'OPEN',
  now()
);
```

16. Step 15 - NOC Creates Force-Sync

Backoffice loads open mismatches:

```
GET /api/provisioning/reconciliation-items?status=OPEN&severity=HIGH
```

NOC decides BSS is correct and network must be updated. It creates:

```
POST /api/provisioning/force-sync-requests
Content-Type: application/json

{
  "sourceItemId": "pri-20260531-000001",
  "subscriptionId": "SUB-700045",
  "serviceRef": "svc_INET_100M_GPON",
  "targetCode": "HUAWEI_NCE_GPON_KE",
  "syncDirection": "BSS_TO_NETWORK",
  "requestedAction": "REAPPLY_PROFILE",
  "reason": "Network profile is still 50M after customer upgrade to 100M."
}
```

Insert:

```
insert into provisioning_force_sync_request (
  force_sync_id,
  operator_code,
  source_item_id,
  subscription_id,
  service_ref,
  target_id,
  sync_direction,
  requested_action,
  status,
  requested_by_user_id,
  reason,
  created_at,
  updated_at
) values (
  'pfs-20260531-000001',
  'WIK',
  'pri-20260531-000001',
  'SUB-700045',
  'svc_INET_100M_GPON',
  'tgt-huawei-nce-ke',
  'BSS_TO_NETWORK',
  'REAPPLY_PROFILE',
  'PENDING_APPROVAL',
  'usr-noc-001',
  'Network profile is still 50M after customer upgrade to 100M.',
  now(),
  now()
);
```

17. Step 16 - Approve And Execute Force-Sync

Supervisor approves:

```
POST /api/provisioning/force-sync-requests/pfs-20260531-000001/approve

{
  "approvalComment": "BSS desired state confirmed from subscription and package history."
}
```

Then execute:

```
POST /api/provisioning/force-sync-requests/pfs-20260531-000001/execute
```

PROV-INT creates a normal provisioning command:

```
idempotency_key = FORCE_SYNC:pfs-20260531-000001
```

```
source_module = PROV-INT
command_type = MODIFY_SERVICE
payload = desired profile from provisioning_desired_state
```

When that command succeeds:

1. Mark force-sync request `COMPLETED`.
2. Mark reconciliation item `RESOLVED`.
3. Emit `ProvisioningForceSyncCompleted`.

18. Suggested Worker Commands

These are Laravel commands or queue worker jobs. They can run in consolidated worker pods.

```
prov:dispatch-pending-commands
prov:retry-failed-commands
prov:poll-async-command-status
prov:reconcile-target --target=HUAWEI_NCE_GPON_KE --scope=REGION:NRB
prov:close-stale-reconciliation-runs
prov:execute-approved-force-sync
```

Recommended schedule:

Command	Frequency
`prov:dispatch-pending-commands`	Continuous queue worker or every minute.
`prov:retry-failed-commands`	Every minute.
`prov:poll-async-command-status`	Every 2 minutes.
`prov:reconcile-target`	Nightly per target, plus manual run.
`prov:close-stale-reconciliation-runs`	Every hour.
`prov:execute-approved-force-sync`	Every minute or event-driven.

19. Error Codes

Error code	Meaning
`PROV_IDEMPOTENCY_REQUIRED`	Command request has no idempotency key.
`PROV_COMMAND_IN_PROGRESS`	Same idempotency key is already dispatching.
`PROV_UNKNOWN_PROVISIONER_KEY`	No active adapter config for provisioner key.
`PROV_TARGET_INACTIVE`	Target exists but is not active.
`PROV_COMMAND_NOT_SUPPORTED`	Target does not support command type.
`PROV_TARGET_TIMEOUT`	Vendor target timed out.
`PROV_VENDOR_REJECTED`	Vendor rejected the command.
`PROV_RETRY_EXHAUSTED`	Command reached max retries.
`PROV_RECON_FETCH_FAILED`	Reconciliation could not fetch observed state.
`PROV_FORCE_SYNC_APPROVAL_REQUIRED`	Force-sync needs approval before execution.

20. Minimum Implementation Order

Build in this order:

1. Migrations for target, adapter config, command, command attempt.
2. Adapter interface and adapter registry.
3. Command API with idempotency.
4. One GPON adapter stub or real integration.
5. Dispatcher and attempt recording.
6. FUL-03 integration for activation commands.

7. FUL-04 integration for restriction/suspension commands.
8. FUL-05 integration for revoke commands.
9. Desired state table update.
10. Reconciliation run and observed state fetch.
11. Reconciliation item comparison.
12. Backoffice read APIs.
13. Force-sync request, approval, execute flow.
14. Metrics and alerts.

This order lets implementation teams prove activation command dispatch before adding the NOC repair features.

21. Test Scenarios

21.1 Activation Command Success

Input:

- FUL-03 sends `ACTIVATE_SERVICE` for internet service.

Expected:

- `provisioning_command` row created.
- One attempt row created.
- Command status `SUCCEEDED`.
- Desired state `ACTIVE`.
- FUL-03 receives success with `commandId`.

21.2 Duplicate Activation Command

Input:

- Same idempotency key sent twice.

Expected:

- One command row.
- One vendor call.
- Second request returns existing result.

21.3 Retryable Timeout

Input:

- Vendor times out on first attempt.

Expected:

- Attempt status `TIMEOUT`.
- Command status `TIMED_OUT` or `FAILED_RETRYABLE`.
- Retry worker sends second attempt later.

21.4 Restriction Command

Input:

- FUL-04 sends `APPLY_RESTRICTION`.

Expected:

- Command sent to correct adapter.

- Desired state becomes `RESTRICTED`.
- Adapter refs are returned to FUL-04.

21.5 Termination Command

Input:

- FUL-05 sends `REVOKE_SERVICE`.

Expected:

- Command sent to correct adapter.
- Desired state becomes `NOT_PRESENT`.
- FUL-05 receives success.

21.6 Reconciliation Profile Mismatch

Input:

- Desired speed is 100M.
- Observed speed is 50M.

Expected:

- Reconciliation item `PROFILE_MISMATCH`.
- Severity `HIGH`.
- Backoffice can create force-sync request.

21.7 Force-Sync

Input:

- Approved `BSS_TO_NETWORK` force-sync request.

Expected:

- New command with `source_module = PROV-INT`.
- Command succeeds.
- Force-sync request `COMPLETED`.
- Reconciliation item `RESOLVED`.

22. Developer Checklist

Before marking the module complete:

- Commands are stored before dispatch.
- Idempotency prevents duplicate vendor calls.
- Attempts are recorded for every vendor call.
- Retry worker respects max retry count and backoff.
- Adapter secrets are not in DB rows.
- FUL-03, FUL-04, and FUL-05 can store command ids in their own outcome tables.
- Desired state is updated after successful commands.
- Reconciliation can fetch observed state from at least one target.
- Mismatches are visible through API.
- Force-sync requires RBAC and audit.
- Destructive force-sync actions can require EM-CFG-04 approval.